

CoS-SSA: SSA for Context-Sensitive Interprocedural Analysis

B.Tech. Thesis Project Report

Devansh Jain
(190100044)

Supervisor:
Prof. Uday Khedker



Department of Computer Science & Engineering
Indian Institute of Technology Bombay
Mumbai 400076 (India)

April 26, 2023

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I declare that I have properly and accurately acknowledged all sources used in the production of this report. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Devansh Jain
(190100044)
April 26, 2023

Acknowledgements

I want to thank my supervisor, Prof. Uday Khedker, for giving me the opportunity to work on such a challenging and intriguing project and for his continuous guidance and engagement via regular formal discussions throughout the project. I would also like to thank Aditya Pradhan, Manjusree M P, Supriya Bhide and other members of the research group for their valuable feedback and suggestions.

Contents

Acknowledgements	ii
Abstract	v
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Static Single Assignment (SSA)	1
1.2 Need of Interprocedural SSA (ISSA)	2
1.3 Goals of this project	3
1.4 Organization of the report	3
2 Context-Sensitive Static Single Assignment (CoS-SSA)	4
2.1 Program representation	4
2.2 Mathematical Formalization	4
2.2.1 Calling Context	4
2.2.2 Metamorphic Assignment	4
2.2.3 Qualified Definition	5
2.2.4 Data Dependence	5
2.3 Motivating Example	5
3 Visualization framework for CoS-SSA	9
3.1 Lexical Analyzer	9
3.2 Semantic Analyzer	9
3.3 Validity Checker	9
3.4 Visualizer	10
4 Value-sensitive Contexts-based Interprocedural Analysis	11
4.1 Interprocedural Data-flow Analysis	11
4.2 Top-down Forward Analysis with Value Contexts	12
4.3 Related Work	12
4.3.1 Optimization	12
4.3.2 Proof of Termination and Complexity Bound	12

5	Design of Value Contexts	13
5.1	Constraints	13
5.2	Targetting Precision	13
5.2.1	Reaching Definitions	13
5.2.2	Reaching Dependence Chains	14
5.2.3	Reaching Dependence Chains with Downwards Exposed QDefs	14
5.2.4	Reaching Dependence Graphs with Downwards Exposed QDefs	15
5.3	Targetting Scalability	15
5.3.1	Set of Reaching Origins	15
5.3.2	Tuple of Reaching Origins	16
5.4	Targetting Precision and Scalability	17
5.4.1	Set of Reaching Dependence	17
5.5	Proof of Correctness	17
6	Implementation of Algorithm	18
6.1	Fixed Point Computations	18
6.1.1	Assumptions	18
6.1.2	Fixed Point Equations	19
6.1.3	Domains	20
6.1.4	Initialization	20
6.1.5	Observations	20
7	Conclusion and Future work	21
	References	22

Abstract

Static Single Assignment (SSA) is an intermediate representation (IR) of a source program that has the property that every scalar variable (whose address is not taken) is defined only once and its uses is dominated by this definition. SSA form is a predominant technology in modern compilers, enabling efficient and powerful sparse optimizations using Def-Use edges instead of analyzing the whole Control-Flow Graph (CFG). However, the scope of SSA is limited because it is only used at an intraprocedural level, i.e., construction of SSA is limited to each procedure. As a long-term project in IIT Bombay, an Interprocedural SSA (ISSA) form is being developed that extends the scope of SSA to an interprocedural level while simultaneously keeping the context-sensitive information; we call this extension of SSA as Context-Sensitive SSA (or CoS-SSA). My role in this project is to help the research group in the visualization of CoS-SSA intuitions and motivating examples, the mathematical formalization of CoS-SSA, and the design of algorithm for the construction of CoS-SSA. In this project report, we describe the motivation and formalization of CoS-SSA, the visualization tool and the design of an algorithm for CoS-SSA construction.

List of Figures

1.1	CFG with large number of Def-Use edges	1
1.2	SSA form of the CFG in Fig. 1.1	2
2.1	CFG for the motivating example	5
2.2	CoS-SSA for the motivating example	6
2.3	Interprocedural Constant Propagation on CoS-SSA in Fig. 2.2	7
2.4	Interprocedural Dead Code Elimination on CoS-SSA in Fig. 2.3	8
5.1	CFG for an example which shows need of Downwards Exposed QDefs	14
5.2	CFG for an example which exposes the flaw in Set of Reaching Origins	15
5.3	Context Transition Graph for Set of Reaching Origins on CFG in Figure 5.2	16
5.4	Context Transition Graph for Tuple of Reaching Origins on CFG in Figure 5.2 . . .	16

List of Tables

2.1	Context-sensitive Data dependence from CFG in Fig. 2.1	6
2.2	Context-sensitive Data dependence from CoS-SSA in Fig. 2.2	6

Chapter 1

Introduction

Program analysis is the process of analysing the behavior of a program to gather information on program correctness and program optimization. These analyses can be done statically without program execution or dynamically during runtime or using both modes. *Data flow analysis* [2] is a static program analysis technique that captures information about the values at each point of the program and how they change over potential control flow paths. The compiler uses these analyses to perform various optimizations which improve the performance of the target program. Some optimizations require the information regarding various definitions of a variable and their uses. Def-use edges are used to propagate data flow information. A def-use edge may bypass a path through a number of control-flow edges and directly connect a definition with its use.

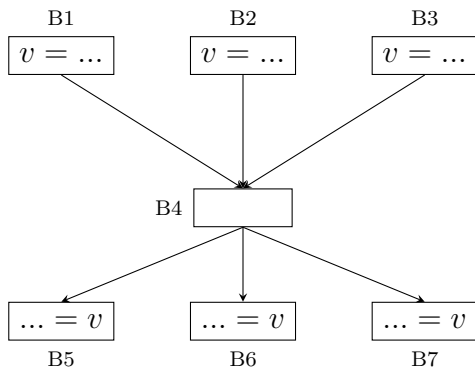


Figure 1.1: CFG with large number of Def-Use edges

However, the number of Def-Use edges can be very large in case of programs with complex control-flow graphs (CFG). m definitions reaching each of n uses result in $m \times n$ def-use edges. Consider a simple example of CFG shown in Fig. 1.1, there are 3 definitions of variable v that are reaching at the entry of the basic block 4. All of these definitions also reach at the entry of the basic blocks 5, 6 and 7, leading to a total of 9 Def-Use edges.

1.1 Static Single Assignment (SSA)

A simple observation from the example (Fig. 1.1) is the possibility of inserting a definition $v = v$ in the basic block 4. Based on this observation, the *Static Single Assignment* (SSA) [1] [2] form was

proposed.

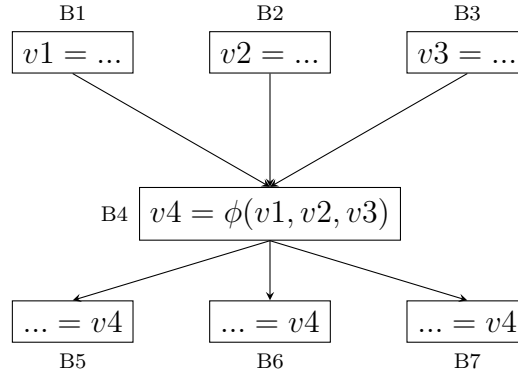


Figure 1.2: SSA form of the CFG in Fig. 1.1

The SSA form is a transformation of the original program that preserves the semantics of the original program. SSA creates different versions of the variables such that each version is defined exactly once, exactly one version reaches the use of a variable and the definition of this version dominates its uses. To ensure that exactly one version reaches every use, ϕ -functions are introduced. A ϕ -function (or ϕ -instruction) is used to merge multiple incoming versions of a variable at the control-flow merge point. For example, in Fig. 1.2, a ϕ -instruction is present at the basic block 4 which merges the SSA versions $v1$, $v2$ and $v3$ of variable v and assigns the result to a new SSA version $v4$.

Two major advantages of SSA form are:

- (i) SSA form reduces the number of def-use edges. In the example (Fig. 1.2), the number of def-use edges is reduced from 9 to 6. Program with m definitions reaching each of n uses with $m \times n$ def-use edges can be transformed to SSA form with $m + n$ def-use edges.
- (ii) Since exactly one version reaches the use of a variable, SSA def-use edges provide *referential transparency* which ensures that the value of a variable does not depend on the location of its use and this enables *sparse analysis*.

1.2 Need of Interprocedural SSA (ISSA)

Modern compilers like LLVM and GCC construct *intraprocedural* SSA form (construction of SSA is limited to each procedure) for a given source program and then perform several optimizations on it. These optimizations are *intraprocedural* in nature and do not consider the effect of the changes made by other procedures. This affects potential optimizations that can be performed across call sites and over multiple procedures. Also, this *intraprocedural* SSA form restricts the optimizations to only scalar variables whose address is not taken. It is not sufficient to perform optimizations on global variables and pointers. This motivates us towards extending the SSA representation at an *interprocedural* level.

The existing methods to construct the Interprocedural SSA (ISSA) do not maintain full context sensitivity and as a result, the client analysis has to do a lot of work to get the context-sensitive information from the ISSA representation. The goal of our group at IIT Bombay is to develop an ISSA representation that is fully flow and context sensitive. We call this representation as the Context-Sensitive Static Single Assignment (or CoS-SSA).

1.3 Goals of this project

The goal of this project is to implement CoS-SSA over the LLVM infrastructure. This CoS-SSA would provide the context-sensitive information to the client analyses.

The first phase of the project consists of mathematical formalization of CoS-SSA for global variables. To assist intuitive understanding of CoS-SSA definition, we developed a prototype to visualize CFG and corresponding CoS-SSA. This visualization tool has been used to design motivating examples which will be used to test the correctness of algorithm of the construction of CoS-SSA. The second phase of the project consists of designing a top-down interprocedural algorithm for the construction of CoS-SSA. The third phase consists of implementing the algorithm over an abstraction of LLVM IR called Simplified LLVM IR Modeling (SLIM). The fourth phase of the project is to extend CoS-SSA representation to handle pointer variables.

1.4 Organization of the report

In chapter 2, we discuss mathematical formalization of CoS-SSA, an interprocedural SSA representation for global variables that is fully flow and context-sensitive, with an example of interprocedural optimization that can be done using the context-sensitive information provided by CoS-SSA. We also discuss how to handle recursive procedures. In chapter 3, we discuss the prototype tool used to visualize CFG and corresponding CoS-SSA. In chapter 4, we discuss top-down interprocedural analysis using value-contexts. We also propose optimization and rough proofs for complexity bounds of the analysis. In chapter 5, we discuss various value-context abstraction choices made while designing design of the top-down interprocedural algorithm for the construction of CoS-SSA. In chapter 6, we discuss the algorithm represented in form of fixed point equations. In chapter 7, we conclude the report and discuss the future work.

Chapter 2

Context-Sensitive Static Single Assignment (CoS-SSA)

In this chapter, we discuss mathematical formalization of CoS-SSA, an interprocedural SSA representation for global variables that is fully flow and context-sensitive.

2.1 Program representation

For simplicity of exposition, we assume that procedures are parameterless and contain only global variables (local variables can be handled by intraprocedural SSA), and are represented by control flow graphs in which a node contains a 3-address code statement. A use statement (such as “*print x*”) is modelled as an assignment “ $\boxed{\text{u}} = x$ ”. An input statement (such as “*scan x*”) is modelled as an assignment “ $x = \boxed{\text{i}}$ ”. $\boxed{\text{u}}$ and $\boxed{\text{i}}$ are synthetic variables that do not appear in the source program.

2.2 Mathematical Formalization

2.2.1 Calling Context

A *context* represents a list of dynamically active procedures and can be represented using a call string which is a sequence of the node numbers of call sites that represent unfinished calls. Calling context can be abstracted to value context, merging contexts that are equivalent in terms of associated data flow value.

2.2.2 Metamorphic Assignment

CoS-SSA introduces metamorphic assignments (aka MMAs) for every node. An MMA for node n consists of a collection of derived forms of the assignment, called *dforms*, indexed by an abstract context α . A *dform* for the assignment of node n in context α , denoted ∂_n^α , is context-sensitive and is obtained by substituting the variables in the RHS of n by the SSA versions of their definitions reaching n in α . Each *dform* introduces a separate SSA version of the LHS variable that is defined exactly once across all procedures. The abstract context α could be a set of call strings (used in illustrations), or a suffix of a call string (used in bottom-up approach), or value contexts (used

in top-down approach). It is important to note that the dforms in an MMA are distinguished by different (abstract) contexts whereas the arguments of ϕ -functions are distinguished by different control flow paths within the same (abstract) context.

2.2.3 Qualified Definition

An assignment in CFG $n : x = y \text{ op } z$ in node n reads variables y and z and defines variable x . The definition corresponding to this statement is denoted by x_n .

In CoS-SSA, we call the definition of x in a dform ∂_n^α as a *qualified* definition (aka qdef), and denote it by x_n^α ; thus dform ∂_n^α has the form $\partial_n^\alpha : x_n^\alpha = y_m^{\alpha'} \text{ op } z_l^{\alpha''}$.

2.2.4 Data Dependence

The def-use edges are analogous to those in SSA, except that they are CoS-SSA qualified definitions. For example, def-use edges corresponding to $\partial_n^\alpha : x_n^\alpha = y_m^{\alpha'} \text{ op } z_l^{\alpha''}$ are $y_m^{\alpha'} \rightarrow x_n^\alpha$ and $z_l^{\alpha''} \rightarrow x_n^\alpha$. The corresponding data dependence is represented as $x_n^\alpha \mapsto \{y_m^{\alpha'}, z_l^{\alpha''}\}$

2.3 Motivating Example

We illustrate the CoS-SSA representation using a simple example. We take a CFG (Fig. 2.1) of a program with two procedures, *main* and *P*. The program has three global variables, t , w and x . We describe the intuition behind its corresponding CoS-SSA representation (Fig. 2.2) and then discuss the context-sensitivity of the data dependence information provided by CoS-SSA.

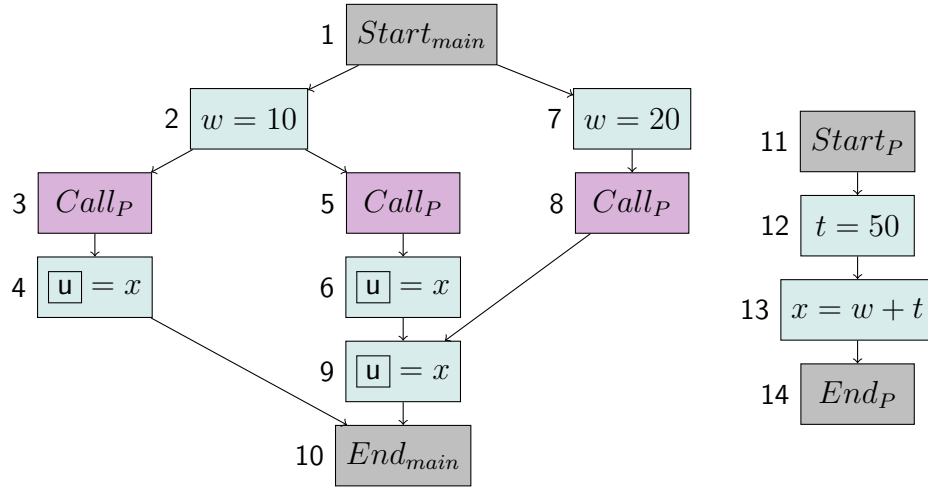


Figure 2.1: CFG for the motivating example

Node	Interprocedurally valid path	Data dependence in CFG
13	(1, 2, 3 , 11, 12, 13)	$x_{13} \mapsto \{w_2, t_{12}\}$
13	(1, 2, 5 , 11, 12, 13)	$x_{13} \mapsto \{w_2, t_{12}\}$
13	(1, 2, 8 , 11, 12, 13)	$x_{13} \mapsto \{w_7, t_{12}\}$
4	(1, 2, 3 , 11, 12, 13, 14, 4)	$u_4 \mapsto \{x_{13}\}$
6	(1, 2, 5 , 11, 12, 13, 14, 6)	$u_6 \mapsto \{x_{13}\}$
9	(1, 2, 5 , 11, 12, 13, 14, 6, 9)	$w_9 \mapsto \{x_{13}\}$
9	(1, 2, 8 , 11, 12, 13, 14, 9)	$w_9 \mapsto \{x_{13}\}$

Table 2.1: Context-sensitive Data dependence from CFG in Fig. 2.1

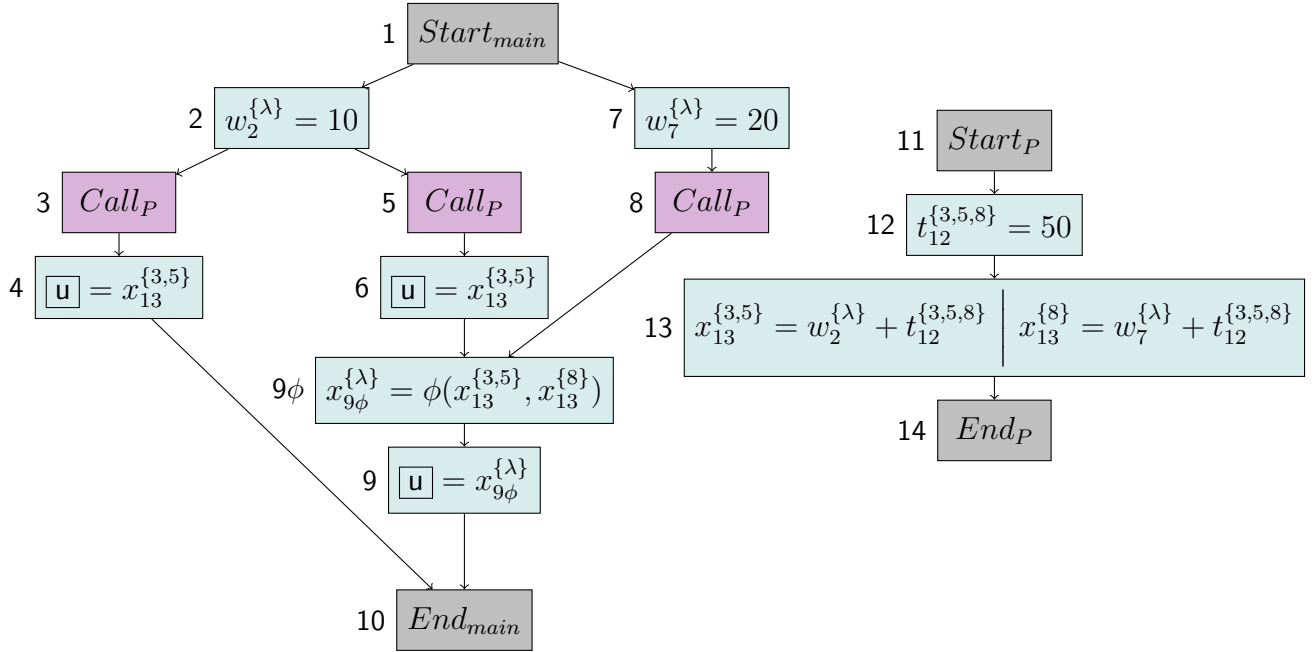


Figure 2.2: CoS-SSA for the motivating example

Node	Interprocedurally valid path	Data dependence in CoS-SSA
13	(1, 2, 3 , 11, 12, 13)	$x_{13}^{\{3,5\}} \mapsto \{w_2^{\{\lambda\}}, t_{12}^{\{3,5\}}\}$
13	(1, 2, 5 , 11, 12, 13)	$x_{13}^{\{3,5\}} \mapsto \{w_2^{\{\lambda\}}, t_{12}^{\{3,5\}}\}$
13	(1, 2, 8 , 11, 12, 13)	$x_{13}^{\{8\}} \mapsto \{w_7^{\{\lambda\}}, t_{12}^{\{8\}}\}$
4	(1, 2, 3 , 11, 12, 13, 14, 4)	$u_4^{\{\lambda\}} \mapsto \{x_{13}^{\{3,5\}}\}$
6	(1, 2, 5 , 11, 12, 13, 14, 6)	$u_6^{\{\lambda\}} \mapsto \{x_{13}^{\{3,5\}}\}$
9 ϕ	(1, 2, 5 , 11, 12, 13, 14, 6, 9 ϕ)	$x_{9\phi}^{\{\lambda\}} \mapsto \{x_{13}^{\{3,5\}}, x_{13}^{\{8\}}\}$
9 ϕ	(1, 2, 8 , 11, 12, 13, 14, 9 ϕ)	$x_{9\phi}^{\{\lambda\}} \mapsto \{x_{13}^{\{3,5\}}, x_{13}^{\{8\}}\}$
9	(1, 2, 5 , 11, 12, 13, 14, 6, 9 ϕ , 9)	$u_9^{\{\lambda\}} \mapsto \{x_{9\phi}^{\{\lambda\}}\}$
9	(1, 2, 8 , 11, 12, 13, 14, 9 ϕ , 9)	$u_9^{\{\lambda\}} \mapsto \{x_{9\phi}^{\{\lambda\}}\}$

Table 2.2: Context-sensitive Data dependence from CoS-SSA in Fig. 2.2

Intuition behind CoS-SSA

Here, we consider a set of call strings as an abstract context. Procedure *main* has one (empty) call string λ reaching it whereas procedure *P* has three call strings reaching it - 3, 5, and 8. Every node in *main* has the abstract context $\{\lambda\}$ whereas every node in *P* has the abstract context as a subset of $\{3, 5, 8\}$. For node 12, we have a constant assignment which is independent of the calling context, thus the abstract context is $\{3, 5, 8\}$. For node 13, we have different versions of w reaching in different calling contexts - $w_2^{\{\lambda\}}$ in 3 and 5; and $w_7^{\{\lambda\}}$ in 8. So, we generate two dforms for node 13 - $\partial_{13}^{\{3,5\}}$ and $\partial_{13}^{\{8\}}$. At node 9, we have two versions of x reaching in same context λ . Thus a ϕ -node is used to merge these two versions of x - $x_{13}^{\{3,5\}}$ and $x_{13}^{\{8\}}$.

Context-sensitivity of CoS-SSA

Unlike data dependence from CFG (Table 2.1), CoS-SSA (Table 2.2) provides more precise context-sensitive data dependence. In node 13, the calling context is associated to the data dependence in form of abstract context ($\{3, 5\}$ or $\{8\}$) which provides keeps the dependencies for two possible values of x separate. This helps at node 4 and 6 where we know which version of x is reaching them and what were the dependencies of that version.

To illustrate the potential of this precise context-sensitive information, consider Interprocedural Constant Propagation as a client analyses. $x_{13}^{\{3,5\}} = 60$ since $w_2^{\{\lambda\}} = 10$ and $t_{12}^{\{3,5,8\}} = 50$ reach dform $\partial_{13}^{\{3,5\}}$. Similarly, $x_{13}^{\{8\}} = 70$ since $w_7^{\{\lambda\}} = 20$ and $t_{12}^{\{3,5,8\}} = 50$ reach dform $\partial_{13}^{\{8\}}$. Unlike data dependence from CFG, this context-sensitive information helps us to determine that value of x reaching node 4 and 6 is 60. Thus, we can propagate 60 to u_4 and u_6 . Now, if we perform Interprocedural Dead Code Elimination, we can eliminate $w_2^{\{\lambda\}}$, $w_7^{\{\lambda\}}$ and $t_{12}^{\{3,5,8\}}$.

The CoS-SSA after Interprocedural Constant Propagation and Interprocedural Dead Code Elimination are shown in Fig. 2.3 and Fig. 2.4 respectively.

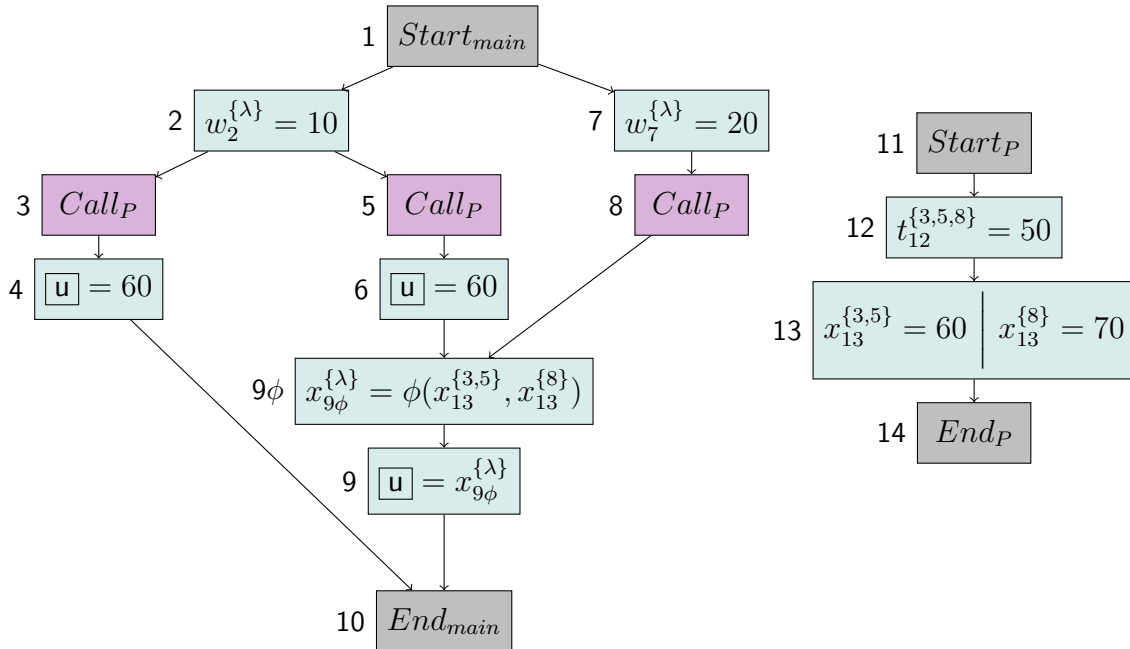


Figure 2.3: Interprocedural Constant Propagation on CoS-SSA in Fig. 2.2

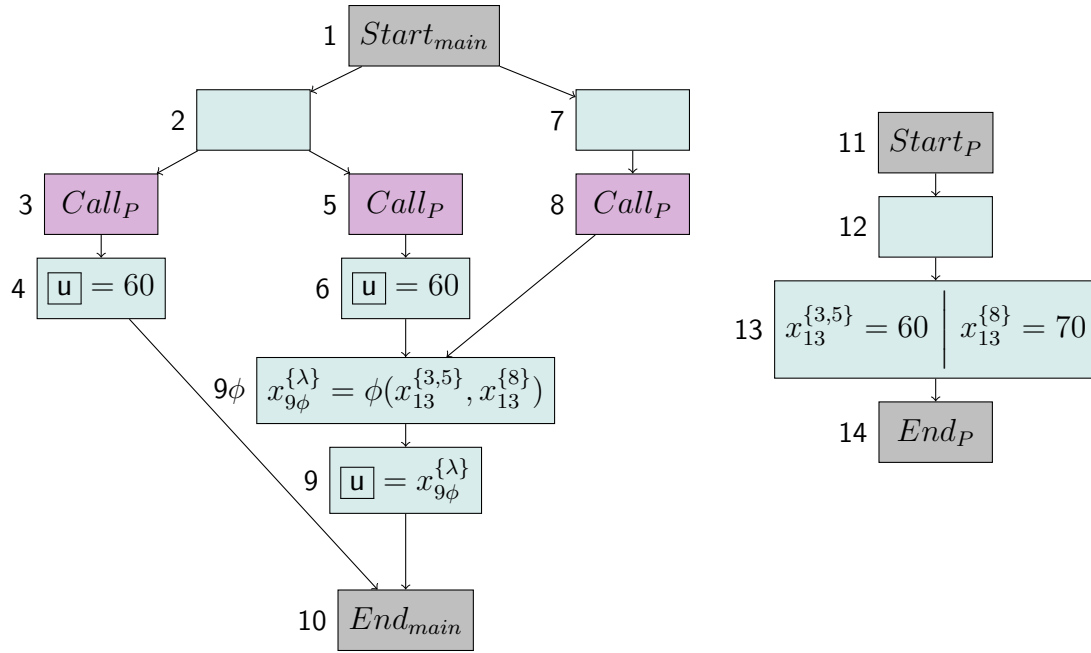


Figure 2.4: Interprocedural Dead Code Elimination on CoS-SSA in Fig. 2.3

Chapter 3

Visualization framework for CoS-SSA

The main goal of this phase is to create a gold reference of examples by intuitively constructing the expected CoS-SSA form. These examples were then to act as regression tests for verifying the mathematical definition of CoS-SSA and checking soundness of algorithm designed for generation of CoS-SSA.

In mathematical definition and algorithm design, the key invariants are the initial Control Flow Graph (CFG), the corresponding Data Dependence Graph (DDG) and the final CoS-SSA with Metamorphic Assignments (SSA¹).

I designed a framework for visualization of CFG, DDG and SSA. The framework was designed to be modular and easy to use. The framework constitutes of Lexical Analyzers, Semantic Analyzers, Validity Checkers, Visualizers for each of the CFG, DDG and SSA.

3.1 Lexical Analyzer

The Lexical Analyzer module is responsible for tokenizing the input file. The module consists of three scanners generated using flex. The input file is a text file containing the CFG, DDG or SSA. The tokens are then passed to the Semantic Analyzer module.

3.2 Semantic Analyzer

The Semantic Analyzer module is responsible for parsing the tokens and creating data structures representing the CFG, DDG or SSA. The module consists of three parsers generated using bison. The data structures are then passed to the Validity Checker module.

3.3 Validity Checker

The Validity Checker module is responsible for checking the validity of the data structures. The Validity Checker module checks for the following:

¹We shall refer to CoS-SSA as SSA for this chapter

3.4 Visualizer

The Visualizer module is responsible for visualizing the CFG, DDG or SSA. The Visualizer module dumps the data structures in the DOT format (graph description language). The DOT format is then passed to the Graphviz tool for visualization.

Chapter 4

Value-sensitive Contexts-based Interprocedural Analysis

4.1 Interprocedural Data-flow Analysis

To construct CoS-SSA, we need to perform interprocedural data-flow analysis to generate the data dependence graph (DDG) of global variables. Various methods of interprocedural data flow analysis can be divided into two broad categories [2]: *functional* approach and *value-based* approach.

A *functional* approach to interprocedural analysis consists of two steps:

1. Designing summary flow functions of a procedure to represent the effects of a call. These functions are context independent and parametrized.
2. Inherited data flow information of a callee is computed by applying the summary flow functions to the calling context of the caller.

These summary flow functions need to be precise, compact, amenable to efficient meet, composition, and application, and most importantly reusable. They are intentional representation of the procedure and computed using a bottom-up traversal over the call graph. However in practice, computation of such summary flow functions is possible only for a limited class of frameworks. This is because constructing summary flow functions requires reducing expressions involving function compositions and intersections. Whether a systematic method of reductions can be devised or not depends on the nature of the flow functions and data flow values.

A *value-based* approach avoids computing summary flow functions. Instead, we directly computed data flow information using a top-down traversal over the call graph. When we encounter a procedure call, we propagate the data flow information to the callee and analyze the callee. This data flow information can be used to abstract set of calling contexts into a single *value-sensitive* context. This approach is more flexible than the functional approach. Conceptually, this approach is simpler than functional approach except that it may have to distinguish between a large number of contexts. Both these approaches for interprocedural analysis inherently handle recursion so long as the frameworks involve finite lattices.

Our group is working on both these approaches. In this thesis, we restrict the discussion to the value-based approach.

4.2 Top-down Forward Analysis with Value Contexts

Generation of DDG requires a forward data-flow analysis of definition and uses of global variables. The fixed point equation for the computations of contexts in forward data-flow analysis is:

$$\begin{aligned}
 Contexts(Q) &= \begin{cases} \{(main, BI)\} & Q = main \\ \{\sigma' \mid \sigma \xrightarrow{m} \sigma' \in Transitions, m : Call_Q\} & otherwise \end{cases} \\
 Transitions &= \{\sigma \xrightarrow{m} (Q, In_m(\sigma)) \mid m : Call_Q, \sigma \in Contexts(ProcOf(m))\} \\
 &\text{where } ProcOf(m) \text{ returns the procedure corresponding to the call instruction } m
 \end{aligned} \tag{4.1}$$

4.3 Related Work

4.3.1 Optimization

Proposal 1: If we have evaluated a procedure for context V_1 as $\top$, then we can reuse the result for a new context V_2 if $V_1 \sqsubseteq V_2$ (V_2 is stronger than V_1). Similarly, if we have evaluated a procedure for context V_1 as $\perp$, then we can reuse the result for a new context V_2 if $V_2 \sqsubseteq V_1$ (V_2 is weaker than V_1).

Proposal 2: If we have evaluated a procedure for context V_1 and encounter a new context V_2 such that $V_2 \sqsubseteq V_1$ (V_2 is weaker than V_1) then the initialization of the internal nodes for evaluating the procedure for V_2 can be same as the final values of the internal nodes upon evaluating for V_1 instead of $\top$.

Minimal Overhead: At every new call site encountered, we do compute the context V_2 and traverse the context list to check if we have computed the procedure for V_2 before or not. Instead of equality operation, we can perform a comparison operation to check if V_2 is a subset of any of the contexts in the list.

4.3.2 Proof of Termination and Complexity Bound

Claim 1. Since the transition edges may be deleted, it doesn't show explicit monotonic behavior. However, the fixed point equations are still monotonic.

[Proof: We can prove this by the fact that transition edges deleted will always be replaced with edges pointing to a context which is strictly stronger than current context. And by DCC (Descending Chain Condition) the fixed point is computable]

[Alt: Take cartesian cross of lattice of edges with 2-value lattice ($\top \equiv$ not deleted edge and $\perp \equiv$ deleted edge). This makes the fixed point equations monotonic.]

Claim 2. It holds true even if we have function pointers since the points-to set always grows which is equivalent to addition of edges.

Chapter 5

Design of Value Contexts

In this chapter, we discuss an algorithm for construction of CoS-SSA for global variables.

5.1 Constraints

In value-sensitive contexts, the data-flow information used in the analysis forms the values used to generate contexts. So, we need to design the lattice of data-flow values such that we don't lose precision - merge contexts which should be differentiated. We also want to achieve compactness - merge contexts which should be merged.

5.2 Targetting Precision

5.2.1 Reaching Definitions

First thought that comes to mind when we think of definitions and uses is reaching definitions. Reaching definitions is a forward data-flow analysis which computes the set of definitions that reach a use. The fixed point equation for reaching definitions is:

$$\begin{aligned} In_n(\sigma) &= \begin{cases} BI & n : Start_{main}, \sigma = (main, BI) \\ \bigcup_{\sigma' \xrightarrow{m} \sigma \in Transitions} In_m(\sigma') & n : Start_Q, Q \neq main \\ \bigcup_{p \in pred(n)} Out_p(\sigma) & otherwise \end{cases} \\ Out_n(\sigma) &= \begin{cases} Out_{End_Q}(\sigma') & n : Call_Q, \sigma \xrightarrow{m} \sigma' \in Transitions \\ (In_n(\sigma) - Kill_n(\sigma)) \cup Gen_n(\sigma) & otherwise \end{cases} \end{aligned} \quad (5.1)$$

Drawback:

A definition x_n reaching an assignment is relevant for the assignment if variable x occurs in the RHS of the assignment. We wish to construct context-sensitive variants of assignments using relevant definitions reaching the assignment in the same context. For non-recursive programs, these distinctions can be made using the reaching definitions of the RHS variables in an assignment. However, for recursive programs, we need to identify a pattern of dependences between definitions

of the RHS variables across recursive calls of the same procedure. Hence, we need to record the transitive effect of definitions and uses across procedure calls. We use reaching dependence chains as a generalization of reaching definitions for this purpose.

An important observation is that reaching definitions are chains of length 0 (no dependence edges are carried during the analysis) and chains of length greater than one edge are useful for recording data dependences via intermediate variables.

5.2.2 Reaching Dependence Chains

We record the transitive effect of definitions and uses, by defining the concept of reaching dependence chains or simply reaching chains, an abstraction of def-use chains in which definitions are explicit and uses are left implicit.

Drawback:

Consider a program (Fig. 5.1) with two procedures, *main* and *P*. The program has two global variables, *a* and *b*.

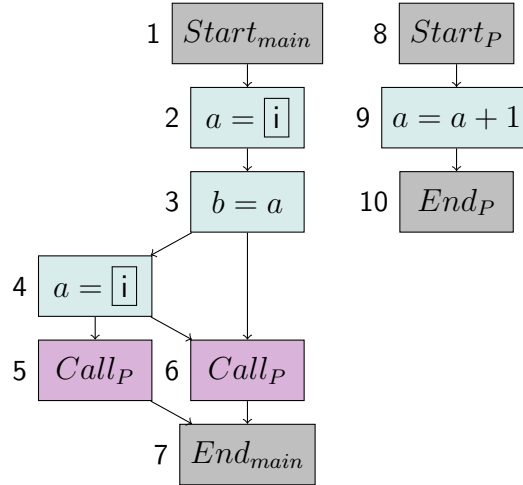


Figure 5.1: CFG for an example which shows need of Downwards Exposed QDefs

The value context for the call string 5 as well as call string 6 would be $\{a_2^{\sigma_0} \cdot b_3^{\sigma_0}, a_4^{\sigma_0}\}$ even though $a_2^{\sigma_0}$ is reaching only when the call string is 6. So, we need to keep track of reaching definition information as well and find a better way to integrate reaching definitions with reaching chains. We can achieve this efficiently by maintaining a list of downwards exposed qdefs for each reaching dependence chain.

5.2.3 Reaching Dependence Chains with Downwards Exposed QDefs

Formally, a dependence chain ψ reaching a given program point is a pair $(seq(\psi), dxdefs(\psi))$ where $seq(\psi)$ is a sequence $d_1 \cdot d_2 \cdots d_k$ and $dxdefs(\psi)$ is a non-empty subset of $\{d_1, d_2, \dots, d_k\}$. If there is no target, the chain is denoted by nil.

Congruence between two chains which is used to differentiate between existing and new value contexts is defined as congruence in the *seq* and *dxdefs* components ignoring the context σ associated with the qdefs.

This definition of context is sound and precise, and thus used as the basis of comparison.

Drawback:

Reaching Dependence Chains hampers scalability of the analysis on large programs due to the overhead of maintaining the chains. Even though robust to recursive procedure, it is not robust to loops within a procedure.

5.2.4 Reaching Dependence Graphs with Downwards Exposed QDefs

To overcome the robustness issues of reaching dependence chains, we decided to use reaching dependence graphs with downwards exposed qdefs. The design is similar to reaching dependence chains with downwards exposed qdefs except that we use a graph instead of a sequence to represent the dependence chain. This idea is inspired from access graphs used in Heap Reference Analysis [2].

Drawback:

Scalability is a concern along with design of data structure to model reaching dependence graphs is challenging.

5.3 Targetting Scalability

5.3.1 Set of Reaching Origins

Now the target was to design the lattice which is scalable without losing soundness and precision. We decided to test if just storing the set of reaching origins for every qdef sufficient for the analysis. This design looks robust to loops within a procedure as well as recursive procedures.

Drawback:

The design fails when we consider assignments with binary expressions in the RHS. Consider a program (Fig. 5.2) with three procedures, *main*, *P* and *Q*. The program has fixe global variables, *a*, *b*, *c*, *x* and *y*.

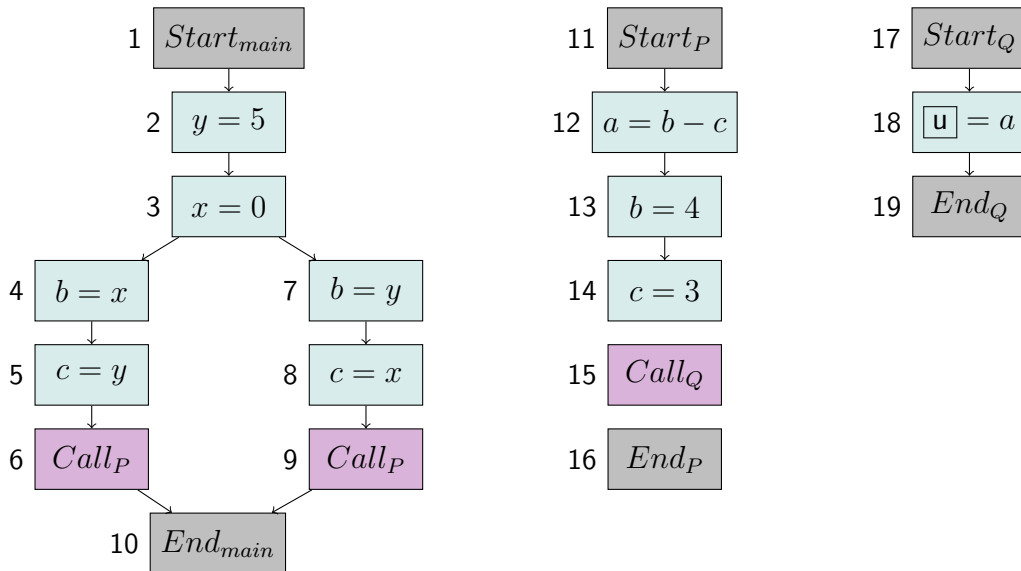


Figure 5.2: CFG for an example which exposes the flaw in Set of Reaching Origins

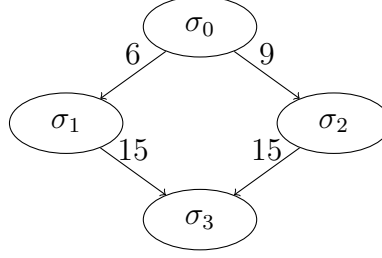


Figure 5.3: Context Transition Graph for Set of Reaching Origins on CFG in Figure 5.2

$$\begin{aligned}
 \sigma_0 &= (main, BI) \\
 \sigma_1 &= (P, \{x_3 \mapsto \emptyset, y_2 \mapsto \emptyset, b_4 \mapsto \{x_3\}, c_5 \mapsto \{y_2\}\}) \\
 \sigma_2 &= (P, \{x_3 \mapsto \emptyset, y_2 \mapsto \emptyset, b_4 \mapsto \{y_2\}, c_5 \mapsto \{x_3\}\}) \\
 \sigma_3 &= (Q, \{a_{12} \mapsto \{x_3, y_2\}, b_{13} \mapsto \emptyset, c_{14} \mapsto \emptyset\})
 \end{aligned} \tag{5.2}$$

Using Set of Reaching Origins, we obtain an imprecise Context Transition Graph (Fig. 5.3 and Eq. 5.2). We can clearly see that call string $6 \cdot 15$ and $9 \cdot 15$ should not be merged, however, they are merged into same value context σ_3 .

5.3.2 Tuple of Reaching Origins

Since the set of reaching origins was failing for binary expressions, we decided to use tuples with size as the number of operands in the RHS of the assignment.

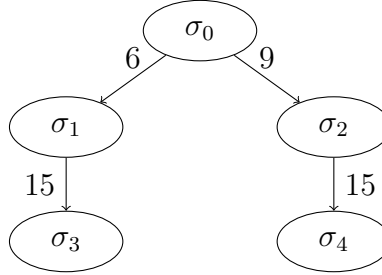


Figure 5.4: Context Transition Graph for Tuple of Reaching Origins on CFG in Figure 5.2

$$\begin{aligned}
 \sigma_0 &= (main, BI) \\
 \sigma_1 &= (P, \{x_3 \mapsto (\emptyset), y_2 \mapsto (\emptyset), b_4 \mapsto (\{x_3\}), c_5 \mapsto (\{y_2\})\}) \\
 \sigma_2 &= (P, \{x_3 \mapsto (\emptyset), y_2 \mapsto (\emptyset), b_4 \mapsto (\{y_2\}), c_5 \mapsto (\{x_3\})\}) \\
 \sigma_3 &= (Q, \{a_{12} \mapsto (\{x_3\}, \{y_2\}), b_{13} \mapsto (\emptyset), c_{14} \mapsto (\emptyset)\}) \\
 \sigma_4 &= (Q, \{a_{12} \mapsto (\{y_2\}, \{x_3\}), b_{13} \mapsto (\emptyset), c_{14} \mapsto (\emptyset)\})
 \end{aligned} \tag{5.3}$$

Using Tuple of Reaching Origins, we obtain a precise Context Transition Graph (Fig. 5.4 and Eq. 5.3). We are able to distinguish between call strings $6 \cdot 15$ and $9 \cdot 15$ as different contexts, σ_3 and σ_4 respectively.

Drawback:

We encountered similar scalability issues with this design as well. The observation was that tuple of

reaching origins had similar structure to reaching dependence chains with intermediate definitions being replaced with their tuple of reaching origins. On top of that, handling assignments with binary expression and copy assignments was challenging while design of contexts. Thus, we decided to pursue the design of reaching dependence chains with downwards exposed qdefs.

5.4 Targetting Precision and Scalability

5.4.1 Set of Reaching Dependence

To overcome scalability concerns, we went back to sets as the data structure to represent reaching dependence chains. The idea is that since we are interested in differentiating definitions which have different dependence graph, we can ignore loops without losing soundness. Similarly, difference in one ancestor is sufficient to distinguish between two definitions. Thus, we just accumulate the dependencies as a set and propagate it.

5.5 Proof of Correctness

Note: $RSet \equiv$ Set of Reaching Dependence without context and $RC \equiv$ Set of Reaching Dependence with context.

To Prove:

- (i) $(RSet1 = RSet2 \Rightarrow RC1 = RC2 [\equiv RC1 \neq RC2 \Rightarrow RSet1 \neq RSet2]) \vee (RC1 \text{ and } RC2 \text{ are same context recursively})$
- (ii) $(RSet1 \neq RSet2 \Rightarrow RC1 \neq RC2 [\equiv RC1 = RC2 \Rightarrow RSet1 = RSet2])$

Goal: It is sound to drop context from the sets which define context.

Proof:

- (i) Non-trivial and work in progress
- (ii) Trivial by design

Chapter 6

Implementation of Algorithm

In this chapter, we discuss implementation of the algorithm for construction of CoS-SSA for global variables.

6.1 Fixed Point Computations

6.1.1 Assumptions

1. All variables in the program are global variables.
2. The program starts with the procedure *main*.
3. Start node of a procedure Q is S_Q .
4. End node of a procedure Q is E_Q .
5. Call node to procedure Q is $Call_Q$.
6. There is no call to the procedure *main*.
7. Syntax for assignment nodes is given by:

$$\begin{aligned} asgn &\longrightarrow x = expr \mid \boxed{u} = x \\ expr &\longrightarrow \boxed{i} \mid \kappa \mid x \mid uop \ x \mid \kappa \ bop \ x \mid x \ bop \ \kappa \mid x \ bop \ y \end{aligned} \tag{6.1}$$

8. $\mathbb{V}$ refers to all global variables in the program.
9. $\mathbb{D}$ refers to the set of all qualified definitions in the program.
10. $\mathbb{P}$ refers to all procedures in the program.
11. $\mathbb{N}$ refers to all nodes in the program.
12. $\mathbb{N}_Q$ refers to all nodes in the procedure Q .
13. $operands(expr)$ refers to set of variables in the expression $expr$.
14. $ProcOf(m)$ refers to the procedure in which the call node m is present.
15. $pred(n)$ refers to the set of nodes that are predecessors of node n .

6.1.2 Fixed Point Equations

$$\begin{aligned}
rdIn_n(\sigma) &= \begin{cases} \{v_0^\sigma \mid v \in \mathbb{V}\} & n : S_{main}, \sigma = (main, \emptyset) \\ \bigcup_{m: Call_Q \sigma' \xrightarrow{m} \sigma \in Transitions} rdIn_m(\sigma') & n : S_Q, Q \neq main \\ \bigcup_{p \in pred(n)} rdOut_p(\sigma) & \text{otherwise} \end{cases} \\
rdOut_n(\sigma) &= \begin{cases} rdOut_{E_Q}(\sigma') & n : Call_Q, \sigma \xrightarrow{n} \sigma' \in Transitions \\ (rdIn_n(\sigma) - rdKill_n) \cup rdGen_n(\sigma) & \text{otherwise} \end{cases} \\
rdGen_n(\sigma) &= \begin{cases} \{x_n^\sigma\} & n : x = expr \\ \emptyset & \text{otherwise} \end{cases} \\
rdKill_n &= \begin{cases} \{x_i^\sigma \mid i \in \mathbb{N}, \sigma \in Contexts\} & n : x = expr \\ \emptyset & \text{otherwise} \end{cases} \\
ddGen_n(\sigma) &= \begin{cases} \{\langle y_m^{\sigma'}, x_n^\sigma \rangle \mid y_m^{\sigma'} \in rdIn_n(\sigma), y \in operands(expr)\} & n : x = expr \\ \{\langle x_m^{\sigma'}, \underline{u}_n^\sigma \rangle \mid x_m^{\sigma'} \in rdIn_n(\sigma)\} & n : \underline{u} = x \\ \emptyset & \text{otherwise} \end{cases} \\
DD_{main} \equiv DD &= \bigcup_{Q \in \mathbb{P}} \bigcup_{\sigma \in Contexts(Q)} \bigcup_{n \in \mathbb{N}_Q} ddGen_n(\sigma) \\
Contexts(Q) &= \begin{cases} \{(main, \emptyset)\} & Q = main \\ \{\sigma' \mid \sigma \xrightarrow{m} \sigma' \in Transitions, m : Call_Q\} & \text{otherwise} \end{cases} \\
Contexts &= \bigcup_{Q \in \mathbb{P}} Contexts(Q) \\
Transitions &= \{\sigma \xrightarrow{m} \sigma' \mid m : Call_Q, \sigma \in Contexts(ProcOf(m)), \\
&\quad \sigma' := (Q, GenContext(rdIn_m(\sigma), DD))\} \\
GenContext(X, DD) &= \{x_n \mapsto RemContext(ReachingQDef(EquivNode(x_n, X), DD)) \\
&\quad \mid x_n \in RemContext(X)\} \\
ReachingQDef(X, DD) &= \{y_m^{\sigma'} \mid \langle y_m^{\sigma'}, x_n^\sigma \rangle \in DD, x_n^\sigma \in (X \cup ReachingQDef(X, DD))\} \\
EquivNode(x_n, X) &= \{x_n^\sigma \mid x_n^\sigma \in X\} \\
RemContext(X) &= \{x_n \mid x_n^\sigma \in X\}
\end{aligned} \tag{6.2}$$

6.1.3 Domains

$$\begin{aligned}
rdIn_n(\sigma) &\subseteq \mathbb{D} \\
rdOut_n(\sigma) &\subseteq \mathbb{D} \\
rdGen_n(\sigma) &\subseteq \mathbb{D} \\
rdKill_n &\subseteq \mathbb{D} \\
ddGen_n(\sigma) &\subseteq \mathbb{D}^2 \\
DD &\subseteq \mathbb{D}^2 \\
Contexts(Q) &\subseteq \{Q\} \times \mathbb{R}^2 \\
Contexts &\subseteq \mathbb{P} \times \mathbb{R}^2 \\
Transitions &\subseteq (\mathbb{P} \times \mathbb{R}^2)^2 \\
GenContext(X, DD) &\subseteq \mathbb{R}^2 \\
ReachingQDef(X, DD) &\subseteq \mathbb{D} \\
EquivNode(x_n, X) &\subseteq \mathbb{D} \\
RemContext(X) &\subseteq \mathbb{R}
\end{aligned} \tag{6.3}$$

Note: $\mathbb{R} = \mathbb{V} \times (\mathbb{N} \cup \{0\})$ and $\mathbb{D} = \mathbb{R} \times Contexts$

6.1.4 Initialization

We compute Least Fixed Point (minimal set) for the above equations. The initialization of the fixed point computation is given by:

$$\begin{aligned}
rdIn_n(\sigma) &= \begin{cases} \{v_0^\sigma \mid v \in \mathbb{V}\} & n : S_{main}, \sigma = (main, \emptyset) \\ \emptyset & \text{otherwise} \end{cases} \\
rdOut_n(\sigma) &= \emptyset \\
DD &= \emptyset \\
Contexts(Q) &= \begin{cases} \{(main, \emptyset)\} & Q = main \\ \emptyset & \text{otherwise} \end{cases} \\
Transitions &= \emptyset \\
ReachingQDef(X, DD) &= \emptyset
\end{aligned} \tag{6.4}$$

6.1.5 Observations

The set of root nodes (nodes with no parents) in DD is given by:

$$\begin{aligned}
root(DD) &= \{v_0^\sigma \mid v \in \mathbb{V}, \sigma = (main, \emptyset)\} \\
&\cup \{x_n^\sigma \mid n : x = \boxed{i}, \sigma \in Contexts(ProcOf(n))\} \\
&\cup \{x_n^\sigma \mid n : x = \kappa, \sigma \in Contexts(ProcOf(n))\}
\end{aligned} \tag{6.5}$$

Therefore, $ReachingQDef(X, DD) = \emptyset$ if $X \subseteq root(DD)$.

Chapter 7

Conclusion and Future work

In this report, we have discussed CoS-SSA, an Interprocedural SSA form that is fully flow and context-sensitive. It provides us context-sensitive and interprocedurally valid def-use information. We explained the mathematical formalization of CoS-SSA with a motivating example.

My main contributions consist of the prototype implementation of CoS-SSA that helps in visualization of CFG and corresponding CoS-SSA, design of the top-down interprocedural algorithm for the construction of CoS-SSA and validating the design choices. I have also worked significantly on proving soundness and precision of the value-context abstraction choices. I helped our group in suggesting and implementing improvements to SLIM IR and VASCO framework which will form the base of the implementation on LLVM infrastructure.

The CoS-SSA formalization for global variables has been completed and we hope to finish the design of algorithm handling scalability and ϕ -nodes and then implementation in the coming months, followed by extension of CoS-SSA to handle pointer variables. We hope to use CoS-SSA in the future for the development of interprocedural sparse optimizations.

References

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., USA, 2006.
- [2] Uday Khedker, Amitabha Sanyal, and Bageshri Sathe. *Data Flow Analysis: Theory and Practice*. CRC Press, 2009.